

Zynq-7000 • 主线 U-Boot • Yocto • kexec • Yosys / nextpnr • REST

ALINX AX7020

开放比特流流水线

从一块空白开发板，到一个接收比特流并将其加载进 FPGA 的 REST 端点：用主线 U-Boot 通过 JTAG 完成板级启动 (bring-up)，在 QSPI 闪存中放入一个能自我更新的 Yocto Linux，用开源工具链生成比特流，再由 systemd 托管 API 服务。不用 SD 卡，不用串口线，整个流程中没有任何厂商工具。

施泓杰 (Stephan Bökelmann, MaxClerkwell)

MaxClerkwell • 德国波鸿 (Bochum) • nabla B

 maxclerkwell.tech

 [@MaxClerkwell](https://twitter.com/MaxClerkwell)

 [@MaxClerkwell](https://www.youtube.com/@MaxClerkwell)

 [Full Stack Engineering](https://github.com/MaxClerkwell)

 [maxclerkwell](https://github.com/maxclerkwell)

 [accelerator-stephan](https://www.linkedin.com/company/accelerator-stephan)

 [@_maxclerkwell](https://www.instagram.com/_maxclerkwell)

 stephan@boekelmann.net

本文档是 maxclerkwell.tech/alinx 五篇系列文章的配套教程。全部源码、配方 (recipe) 与脚本：
github.com/MaxClerkwell/ax7020-bringup。
开发板由 ALINX 提供；厂商对内容没有任何影响。

版本 1.0 (简体中文)

2026 年 9 月 15 日

开发板：AX7020, XC7Z020-2CLG400

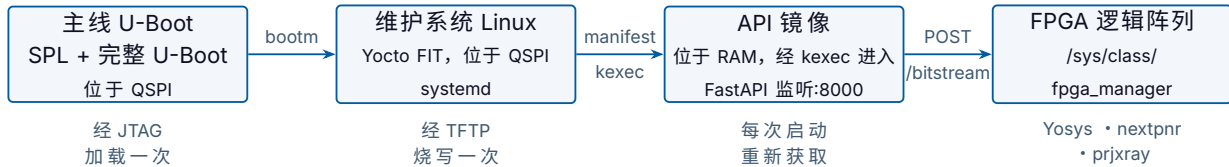
目录

1 本教程要搭建什么	3
1.1 范围	3
1.2 你需要准备什么	3
1.3 全文用到的开发板参数	4
1.4 代码仓库	4
2 主机环境	4
2.1 工具链与工具	4
2.2 JTAG 适配器	4
2.3 实验网络	5
2.4 控制台与 TFTP	5
3 阶段 1: 把主线 U-Boot 写入 QSPI 闪存	5
3.1 四个文件	5
3.2 编译	6
3.3 闪存布局	6
3.4 通过 JTAG 完成板级启动	6
3.5 烧写	7
3.6 独立启动	7
4 阶段 3: 闪存中的 Yocto Linux	7
4.1 层 (layer)	8
4.2 构建	8
4.3 先从 RAM 启动	9
4.4 烧写 FIT	9
5 阶段 2: 镜像服务器与更新器	9
5.1 服务器	9
5.2 发布	10
5.3 更新器	10
6 阶段 4: 不用 Vivado 生成比特流	11
6.1 工具链	11
6.2 设计	11
6.3 流程	11
6.4 通过 SSH 加载	12
7 阶段 5: 把比特流 API 做成镜像	12
7.1 服务	12
7.2 配方 (recipe)	12
7.3 构建、发布并更新维护系统	13
7.4 冷启动	13

8 日常操作	14
8.1 找到开发板	14
8.2 SSH	14
8.3 回滚镜像	14
8.4 重写闪存	14
8.5 备份闪存	14
9 下一步	15
关于作者	16

1 本教程要搭建什么

ALINX AX7020 是一块 Zynq-7000 开发板：两个 Cortex-A9 核心与一块 7 系列 FPGA 逻辑阵列 (fabric) 集成在同一颗芯片上，处理器侧配有 1GiB DDR3、32 MiB QSPI 闪存和千兆以太网。厂商推荐的路线是 Vivado、Xilinx FSBL、PetaLinux 和一张 SD 卡。本教程走另一条路：把上述每一个环节都换成源码可读的组件。



全部完成后，一次上电重启看起来是这样的，板上只接电源和以太网线：

```

U-Boot: DHCP client bound to address 10.42.100.153
U-Boot: ## Loading kernel (any) from FIT Image at 02000000 ...
maintenance system -> ax7020-update.service -> image server -> kexec
API image: GET http://10.42.100.156:8000/state -> {"state":"unknown", ...}
POST blinky.bit -> {"state":"operating", "idcode":"0x03727093", ...}
  
```

1.1 范围

这是一条干净的路径：所有需要的命令，按需要的顺序给出，每一步之后都有一个检查点。各项设计决策背后的理由，以及每一步可能出错的种种方式，都在 maxclerkwell.tech/alinx 的系列文章里。系列文章中的某个事实若是通过某一步所必需的，它会以“规则”的形式出现在这里；若只是有趣，则不会出现。

1.2 你需要准备什么

- 一块 ALINX AX7020，带 12 V 电源和一根以太网线。
- 一个 JTAG 适配器。一块 FTDI FT232H 开发板就够了；这块板子不需要复位线，也不需要缓冲使能线。
- 一台 Linux 工作站（本文使用 Debian 13），带第二个以太网口或 USB 网卡用于搭建私有实验网络，约 100 GB 空闲磁盘空间给 Yocto，以及几个小时的构建时间。
- 公司网络或实验室网络中一台能跑一个 nginx 容器的机器。路由器或 NAS 最合适。
- 不需要 SD 卡、USB-UART 线、Vivado 或 PetaLinux。

1.3 全文用到的开发板参数

SoC	XC7Z020-2CLG400, JTAG IDCODE 0x23727093
DDR3	2 × MT41J256M16, 32 位总线, 533 MHz, 1GiB
QSPI	Winbond W25Q256, 32 MiB, 单片选, MIO 1..6
以太网	PS GEM0, RGMII 位于 MIO 16..27, RTL8211E-VL PHY, 地址 1
启动跳线	J13 : 左 = SD, 中 = QSPI, 右 = JTAG
PL 时钟	50 MHz 振荡器, 引脚 U18
PL LED	M14、M15、K16、J16, 高电平有效, LVCMOS33

1.4 代码仓库

下文引用的每一个文件都在 github.com/MaxClerkwell/ax7020-bringup。本文档中的路径均相对于仓库根目录。

uboot/	DTS, defconfig, ps7_init for the mainline U-Boot tree
openocd/	adapter definition and the JTAG bring-up script
tools/	netconsole shell, TFTP server, publish and flash scripts
yocto/	meta-ax7020 layer and the build setup script
bitstream/	open toolchain flow, blinky and empty designs, bit2bin.py
api/	the FastAPI service
docs/	session walkthroughs (German), longer than this tutorial

2 主机环境

2.1 工具链与工具

```
sudo apt install gcc-arm-none-eabi openocd python3 make git
sudo apt install gawk wget diffstat unzip texinfo gcc build-essential \
    chrpath socat cpio python3-pip python3-pexpect xz-utils debianutils \
    iputils-ping python3-git python3-jinja2 python3-subunit zstd lz4 file \
    locales libacl1
```

第二行是 Yocto 在 Debian 上的主机依赖列表。注意这里是 `lz4` 而不是旧的 `liblz4-tool`；Yocto 仍然按旧名字 `lz4c` 查找该二进制文件，[yocto/hostbin/lz4c](#) 是一个符号链接，无需 root 权限即可解决这个问题。

2.2 JTAG 适配器

[openocd/ft232h.cfg](#) 描述了一块裸 FT232H。在 FTDI 芯片上，MPSSE 引脚分配是硅片固定的 (ADBUS0 = TCK, 1 = TDI, 2 = TDO, 3 = TMS)，所以只需声明空闲电平和方向：

```
adapter driver ftdi
ftdi vid_pid 0x0403 0x6014
ftdi layout_init 0x0008 0x000b
adapter speed 1000
transport select jtag
```

```
openocd -f openocd/ft232h.cfg -f target/zynq_7000.cfg
```

✓ **检查点** 扫描链上显示两个 TAP 和两个核心：

```
JTAG tap: zynq_pl.bs tap/device found: 0x23727093 (Xilinx, part 0x3727)
JTAG tap: zynq.cpu tap/device found: 0x4ba00477 (ARM Ltd)
zynq.cpu0: hardware has 6 breakpoints, 4 watchpoints
zynq.cpu1: hardware has 6 breakpoints, 4 watchpoints
```

2.3 实验网络

板级启动阶段，开发板挂在工作站后面的一个私有网络上。用 NetworkManager 的 共享 (shared) 连接提供 DHCP，同时把开发板隔离在家庭网络之外：

```
nmcli connection add type ethernet ifname <iface> con-name zynq-lab \
    ipv4.method shared ipv4.addresses 192.168.77.1/24 ipv6.method disabled
nmcli connection up zynq-lab
```

开发板拿到租约后，用 `ip neigh show dev <iface>` 查找它的地址。

2.4 控制台与 TFTP

这套方案里没有串口线。U-Boot 的控制台是它的网络控制台 (netconsole)，走 UDP 6666 端口，[tools/ncsh.py](#) 是与之配套的 shell。[tools/tftpd.py](#) 是一个非特权 TFTP 服务器，因此不需要任何系统服务：

```
tools/tftpd.py tftp 6969 & # serves the tftp/ directory
tools/ncsh.py <board-ip> # interactive
tools/ncsh.py <board-ip> "bdinfo" "sf probe 0 30000000 0" # one-shot
```

在开发板上，`setenv tftpdstp 6969` 让 tftpboot 指向这个端口。

3 阶段 1：把主线 U-Boot 写入 QSPI 闪存

主线 U-Boot 不支持 AX7020。四个文件让它认识这块板子；它们镜像在 [uboot/](#) 目录下，需要放进 v2026.10-rc2 或更新版本的 U-Boot 源码树中。

3.1 四个文件

[uboot/zynq-ax7020.dts](#)

[arch/arm/dts/](#)。开发板描述，每个值都由厂商的硬件交接文件 (hardware handoff) 和原理图推导而来。

[uboot/alinx_ax7020_defconfig](#)

[configs/](#)。由 `xilinx_zynq_virt_defconfig` 派生：启用 netconsole、传统网络栈、TFTP put、md5sum；关闭 SPL 的 FIT 加载和重定位的 SPL 栈。

`uboot/ps7_init_gpl.c, .h`

`board/xilinx/zynq/zynq-ax7020/`。Vivado 生成的 PS 初始化代码 (PLL、MIO、DDR 时序)，从厂商的 XSA 归档中提取，XSA 其实就是一个普通 zip。为适配 GCC 15，五处声明加上了 (void)。

再做两处修改即可完成：把 `zynq-ax7020.dtb` 加入 `arch/arm/dts/Makefile`；在 `include/configs/zynq-common.h` 的默认环境中设置 `ncip = 255.255.255.255` 以及 `stdin/stdout/stderr = serial,nc`，这样在环境变量可写之前，`netconsole` 就已经就位。

⚠ 规则 这块板子上有两条设备树 (device tree) 规则是强制的。`bootph-all` 必须同时出现在闪存节点和 QSPI 控制器节点上，否则 SPL 找不到片选。而且 `spi-rx-bus-width` 必须是 <1>：在 SPL 中，四线读 (quad read) 返回的数据会偏移三个字节，而完整的 U-Boot 不会暴露这个故障。

3.2 编译

启动命令直接编译进二进制，而不是存在环境变量里，因为 JTAG 启动模式下 `saveenv` 不起作用，而且过长的 `setenv` 命令行会在 `netconsole` 上断裂。它从 FIT 分区读取 24 MiB：

```
CONFIG_BOOTCOMMAND="sf probe 0 30000000 0; sf read 0x20000000 0x340000 0x1800000; bootm 0x20000000"
```

```
cd u-boot
make alinx_ax7020_defconfig
scripts/config --disable TOOLS_MKEFICAPSULE      # host tool needs gnutls
make olddefconfig
make -j$(nproc) CROSS_COMPILE=arm-none-eabi-
cp spl/boot.bin u-boot.img ../tftp/
```

✅ **检查点** 生成了 `spl/boot.bin` (约 107 kB，带 Xilinx 启动头的 SPL) 和 `u-boot.img` (约 1.1MB，传统 ulimage 格式)。

3.3 闪存布局

偏移	内容	分区大小	mtd
0x000000	<code>boot.bin</code>	1 MiB	0
0x100000	<code>u-boot.img</code>	2 MiB	1
0x300000	环境变量 + 冗余副本	256 KiB	2
0x340000	FIT：内核 + DTB + initramfs	28.75 MiB	3

Linux 设备树中有同一张映射表，所以运行中的 Linux 会把这些分区看作 `/dev/mtd0..3`。

3.4 通过 JTAG 完成板级启动

开发板冷态，J13 拨到 JTAG，上电。`openocd/load-uboot.cfg` 把 SPL 加载到片上内存，等待 `ps7_init` 把 DDR 初始化完成，写入并回读一个 DDR 测试字，把完整 U-Boot 加载到 `0x4000000`，

校验后跳转：

```
openocd -f openocd/ft232h.cfg -f target/zynq_7000.cfg -f openocd/load-uboot.cfg
```

⚠ 规则 只在 SPL 处于停机 (halt) 状态、缓存关闭时加载完整 U-Boot。若在一个先前的完整 U-Boot 停机时向 DDR 加载，其已启用的 D-cache 会以 32 字节为单位破坏镜像。另外，任何一次 QSPI 模式启动之后，再次通过 JTAG 加载前必须在 JTAG 模式下重新上电：ps7_init 不是幂等的。

✔ **检查点** 通过 netconsole，bdinfo 报告 1GiB DRAM，sf probe 0 30000000 0 检测到 w25q256，容量 32 MiB。

3.5 烧写

只能从一个“在 JTAG 模式下刚重新上电、并通过 JTAG 加载”的 U-Boot 执行。大小是文件大小的十六进制表示 (printf "%x\n" \$(stat -c %s tftp/boot.bin))：

```
tools/ncsh.py <ip> "setenv tftpdstp 6969" "sf probe 0 30000000 0" \
  "tftpboot 0x10000000 boot.bin" "tftpboot 0x11000000 u-boot.img" \
  "md5sum 0x10000000 <size_boot_bin>" "md5sum 0x11000000 <size_u_boot_img>"
tools/ncsh.py <ip> "sf erase 0 0x300000" \
  "sf write 0x10000000 0 <size_boot_bin>" \
  "sf write 0x11000000 0x100000 <size_u_boot_img>"
tools/ncsh.py <ip> "sf read 0x12000000 0 <size_boot_bin>" \
  "md5sum 0x12000000 <size_boot_bin>" \
  "sf read 0x13000000 0x100000 <size_u_boot_img>" \
  "md5sum 0x13000000 <size_u_boot_img>"
```

✔ **检查点** 两个回读的 MD5 都与主机上 md5sum 计算的结果一致。在此之前，任何东西都不算烧写成功。

3.6 独立启动

断电，J13 拨到 QSPI，拔掉 JTAG，上电。

✔ **检查点** U-Boot 自行出现在 netconsole 上，且 modeboot = qspiboot。JTAG 适配器可以收进抽屉了；从这里开始，一切都通过以太网完成。

4 阶段 3：闪存中的 Yocto Linux

阶段编号沿用系列文章的顺序，阶段 2 的镜像服务器实际上是在阶段 3 的 Linux 之后才搭建的。

Linux 需要的一切打包成一个制品：一个包含内核、设备树和 initramfs 的 FIT 镜像，写入 FIT 分区。根文件系统位于 RAM 中。这个镜像就是维护系统：体积小，很少变动，负责获取其余的一切。

4.1 层 (layer)

全程使用 scarthgap (Yocto 5.0 LTS)，这是 meta-xilinx 支持的最新分支：

```
cd yocto
for r in "https://git.yoctoproject.org/poky poky" \
        "https://github.com/Xilinx/meta-xilinx meta-xilinx" \
        "https://git.yoctoproject.org/meta-arm meta-arm" \
        "https://github.com/openembedded/meta-openembedded meta-openembedded"; do
    set -- $r; git clone --depth 1 -b scarthgap "$1" "$2"
done
```

[yocto/meta-ax7020](#) 是开发板层。其组成部分如下，路径相对于该层：

[conf/machine/ax7020.conf](#)

引用 meta-xilinx 的 zynq-generic，把 KERNEL_IMAGETYPE 设为 fitImage，不声明任何 bootloader 提供者，并指定 ax7020-initramfs 作为放进 FIT 的 initramfs。

[recipes-kernel/linux/files/zynq-ax7020.dts](#)

Linux 设备树，包含闪存分区映射。

[recipes-kernel/linux/files/ax7020.cfg](#)

内核配置片段：Zynq FPGA manager、FPGA region 与 overlay、netconsole、MTD 与 QSPI、kexec；关闭图形、声音、无线和 IPv6。

[recipes-core/images/ax7020-initramfs.bb](#)

维护镜像：core-image-minimal 打包为 cpio.gz，外加 dropbear、curl、kexec-tools、更新器和操作员的 SSH 公钥。

[recipes-support/ax7020-ssh-key](#)

把 `authorized_keys` 安装到 `${ROOT_HOME}`。构建前，把你的公钥放到 [yocto/meta-ax7020/recipes-support/ax7020-ssh-key/files/authorized_keys](#)。

[yocto/setup-build.sh](#)

从版本控制中重新生成 [build/conf](#)：machine 为 ax7020，INIT_MANAGER = "systemd"，Yocto 不构建 bootloader。

4.2 构建

```
source yocto/setup-build.sh
setsid nohup bitbake virtual/kernel > build.log 2>&1 &      # detached: a killed
tail -f build.log                                           # client kills the build
```

首次构建需要几个小时。FIT 输出在 [yocto/build/tmp/deploy/images/ax7020/](#) 下，文件名为 `fitImage-ax7020-initramfs-ax7020-ax7020`；把它复制到 `tftp/fitImage`。

✔ **检查点** `dumpimage -l tftp/fitImage` 列出一个配置，包含内核、DTB `zynq-ax7020.dtb` 和一个 `ramdisk`，每一项都带 SHA256 哈希。FIT 小于 24 MiB。

4.3 先从 RAM 启动

从一个通过 JTAG 加载的 U-Boot 开始，在写入任何东西之前：

```
tools/ncsh.py <ip> "setenv tftpdstp 6969" "tftpboot 0x2000000 fitImage" "bootm 0x2000000"
```

✔ **检查点** Linux 启动，以主机名 `ax7020` 获取 DHCP 租约，并且 **22 端口有响应**，仅允许密钥登录。ping 不能作为判据；一个网络栈正常的 U-Boot 同样会响应 ping。

4.4 烧写 FIT

```
tools/ncsh.py <ip> "setenv tftpdstp 6969" "tftpboot 0x2000000 fitImage" \
"sf probe 0 30000000 0" "sf erase 0x340000 0x1CC0000" \
"sf write 0x2000000 0x340000 <size_fit>" \
"sf read 0x3000000 0x340000 <size_fit>" "md5sum 0x3000000 <size_fit>" \
"md 0x3000000 4" "md 0x3800000 4"
```

✔ **检查点** 回读的 MD5 一致，并且回读数据中两个不同偏移处的字彼此不同。一块地址线错位的闪存可能返回一份自治却错误的镜像，单一校验和是查不出来的。

断电，J13 拨到 QSPI，上电。

✔ **检查点** Linux 自行从闪存启动，可以通过 SSH 访问。此时开发板可以从实验网络移到公司网络；它会以 `ax7020` 的名字向 DHCP 报到。

5 阶段 2：镜像服务器与更新器

维护系统从不改动闪存中的内容。每次启动时，它向一个 URL 请求清单 (manifest)，下载内核、设备树和 `initramfs`，校验之后 `kexec` 进入结果。一个坏掉的镜像只需要重新上电，而不必重新烧写，因为重新上电后维护系统就回来了。

5.1 服务器

在开发板能访问到的一台机器上跑一个 `nginx` 容器，只读，只绑定 LAN 地址，提供一个目录：

```
docker run -d --name ax7020-image-server --restart unless-stopped \
-p 10.42.0.1:8080:80 \
-v /srv/ax7020-images:/usr/share/nginx/html:ro \
-v /etc/ax7020-image-server/default.conf:/etc/nginx/conf.d/default.conf:ro \
--read-only --tmpfs /var/cache/nginx --tmpfs /var/run --tmpfs /tmp nginx:alpine
```

`default.conf` 中包含 `autoindex on;` 和 `disable_symlinks off;`。每次构建得到一个带时间戳的目录；`ax7020-latest` 是指向最新目录的符号链接。开发板只知道一个 URL：

```
http://10.42.0.1:8080/ax7020-latest/manifest
```

5.2 发布

`tools/publish-image.sh` 通过 `scp` 把 `zImage`、`DTB` 和 `cpio.gz` 从 Yocto 的 `deploy` 目录复制过去，写入带 SHA256 校验和的清单，然后重新指向符号链接：

```
tools/publish-image.sh AI-heimdall:/srv/ax7020-images # API image
tools/publish-image.sh -i ax7020-initramfs AI-heimdall:/srv/ax7020-images #
maintenance image
```

```
kernel zImage          542ef58f...
dtb     zynq-ax7020.dtb 1e3fa9c7...
initrd  initramfs.cpio.gz 4e73ba34...
```

⚠ 规则 清单列出的是三个文件，而不是一个 FIT。32 位 ARM 上的 `kexec-tools` 没有 FIT 加载器，而它的 `zImage` 探测逻辑接受任何文件；给它一个 FIT，它会跳进设备树，开发板悄无声息地死掉。更新器会自己检查偏移 `0x24` 处的 `zImage` 魔数和 `DTB` 魔数，因为下游没有任何环节会检查。

5.3 更新器

更新器位于 `yocto/meta-ax7020/recipes-support/ax7020-updater/files/`：`ax7020-update` 是 shell 脚本，`ax7020-update.conf` 设置 `IMAGE_URL`（留空表示留在维护系统），`ax7020-update.service` 每次启动运行它一次：

```
[Unit]
After=network-online.target time-sync.target
Wants=network-online.target

[Service]
Type=oneshot
ExecStartPre=/bin/sleep 15
ExecStart=/usr/bin/ax7020-update
StandardOutput=journal+console
```

脚本获取清单，然后逐个下载文件，比对每个 SHA256，检查两个魔数，执行 `kexec -l zImage --dtb --initrd --command-line`，随后 `kexec -e`。 `DRY_RUN=1` 会在跳转前停下。任何一步失败，`unit` 就失败，开发板仍可通过 SSH 访问。

✅ 检查点 把维护镜像本身发布出去，设置 `IMAGE_URL`，重启：大约二十秒后，同一个系统回来了，运行时间 (uptime) 只有几秒，而 `journalctl -u ax7020-update` 显示三个校验和均已通过。

6 阶段 4：不用 Vivado 生成比特流

6.1 工具链

Zynq-7020 是 7 系列器件，在 openXC7 流程的覆盖范围内。除 Yosys 之外，一切都在 `build/` 下从源码构建：

工具	来源	作用
Yosys 0.66	Debian 软件包	综合, <code>synth_xilinx</code>
nextpnr-xilinx	openXC7 分支	布局布线, 可读取 XDC
prjxray	f4pga	<code>fasm2frames</code> 、 <code>xc7frames2bit</code>
prjxray-db	zynq7	比特流数据库, <code>xc7z020</code> 的 <code>chipdb</code>

XC7Z020 的芯片数据库 (`chipdb`) 由 `prjxray-db` 生成一次；它有几百兆字节，下面的一切都依赖它。

6.2 设计

`bitstream/blinky/blinky.v`：一个由 50 MHz PL 振荡器驱动的 27 位计数器，高位接四个 LED，还有一行不可省略的代码：

```
module blinky (input wire sys_clk, output wire [3:0] led);
    reg [26:0] cnt = 27'd0;
    always @(posedge sys_clk) cnt <= cnt + 27'd1;
    assign led = cnt[26:23];
    (* keep *) PS7 ps7_i();
endmodule
```

⚠ 规则 Zynq 设计**必须实例化 PS7 模块**，即便完全不使用它。这样 `nextpnr` 会把 PS7 所有未使用的输入接地。若没有它，一旦内核使能 PL 到 PS 的电平转换器，逻辑阵列就会驱动直连两个 Cortex-A9 核心的 `nFIQ/nIRQ` 线 (`IRQF2P[19:16]`，绕过中断控制器)。两个核心永远在 FIQ 处理中空转，什么都不会打印出来。

引脚约束来自 `bitstream/blinky/blinky.xdc`，一个普通的 XDC 文件，`nextpnr` 直接读取。

6.3 流程

`bitstream/blinky/Makefile` 串联四个工具和一次格式转换：

```
blinky.v --yosys synth_xilinx--> blinky.json
--nextpnr-xilinx----> blinky.fasm
--fasm2frames-----> blinky.frames
--xc7frames2bit-----> blinky.bit
--bit2bin.py-----> blinky.bin
```

```
cd bitstream/blinky && make blinky.bin
```

最后一步 `bitstream/bit2bin.py` 之所以存在，是因为内核的 `zynq-fpga` 驱动期望 `bootgen` 生成

的布局：去掉 .bit 的 ASCII 头，每个 32 位字做字节交换。

✔ **检查点** `xxd -l 16 blinky.bin` 显示 `ffff ffff ffff ffff 6655 99aa`，即字节交换后的同步字。

6.4 通过 SSH 加载

FPGA manager 接受的是相对于 `/lib/firmware` 的文件名：

```
ssh root@<ip> 'mkdir -p /lib/firmware; cat > /lib/firmware/blinky.bin' < blinky.bin
ssh root@<ip> 'echo blinky.bin > /sys/class/fpga_manager/fpga0/firmware;
               cat /sys/class/fpga_manager/fpga0/state'
```

`tools/stage4-first-load/load-bitstream.sh` 做同样的事，但把内核日志通过 `netconsole` 重定向到工作站，第一次加载时值得这么做。

✔ **检查点** `state` 读出 `operating`，四个 LED 闪烁，开发板的 22 端口仍然有响应。

7 阶段 5：把比特流 API 做成镜像

7.1 服务

`api/app.py` 是一个 FastAPI 应用，有三个端点：

GET	<code>/state</code>	FPGA manager 状态以及最后加载的内容
POST	<code>/bitstream</code>	multipart file=, .bit 或 .bin, 可选 ?sha256=; 校验哈希, 查找同步字, 将 IDCODE 数据包与 XC7Z020 比对, 转换 .bit, 加载, 待逻辑阵列就绪后返回
DELETE	<code>/bitstream</code>	加载 <code>empty.bin</code> (一个只含 PS7 的设计), 因为 FPGA manager 没有卸载操作

IDCODE 检查在头部查找 type-1 数据包 `0x30018001`，把紧随其后的字掩码到 28 位后与 `0x23727093` 比较；最高四位是硅片版本号。错误以 `{"error": "..."}` 的形式返回，状态码 400 或 500。交互式文档在 `/docs` 提供。

7.2 配方 (recipe)

全部位于 `yocto/meta-ax7020`，路径相对于该层：

`classes/pypi-wheel.bbclass`

把纯 Python 的 wheel 解压到 `site-packages` 完成安装。scarthgap 没有 FastAPI 构建元数据所要求的 `pdm-backend` 对应的 class，而 wheel 本身就已经是安装后的目录树。

`recipes-devtools/python/`

`fastapi` 0.111.1、`starlette` 0.37.2、`uvicorn` 0.30.6、`click` 8.1.7、`python-multipart` 0.0.9 以 wheel 形式安装；`pydantic`、`anyio` 和 `h11` 来自 `meta-python`。

`recipes-devtools/uv/uv_0.12.13.bb`

`astral` 的 `armv7-musl` 静态二进制，哈希已固定，用于在板上做 `uv run --with ...` 实验。不

在服务的启动路径上。

`recipes-support/ax7020-api/ax7020-api.bb`

把 `app.py` 装到 `/opt/ax7020-api`, `empty.bin` 装到 `/lib/firmware`, 并启用下面的 unit。

`recipes-core/images/ax7020-api-image.bb`

`core-image-minimal` 加上以上内容、Python、dropbear 和操作员公钥。INITRAMFS_MAXSIZE 提高到 512 MB。不含更新器。

`yocto/meta-ax7020/recipes-support/ax7020-api/files/ax7020-api.service:`

```
[Service]
Type=simple
WorkingDirectory=/opt/ax7020-api
ExecStart=/usr/bin/python3 -m uvicorn app:app --host 0.0.0.0 --port 8000
Restart=on-failure
RestartSec=3
```

⚠ 规则 API 镜像绝不能包含更新器。一旦设置了 `IMAGE_URL`, 它会在每次启动时下载自己并 `kexec` 进入自己, 无穷无尽。

7.3 构建、发布并更新维护系统

```
source yocto/setup-build.sh
bitbake ax7020-api-image virtual/kernel          # API image + new maintenance FIT
tools/publish-image.sh AI-heimdall:/srv/ax7020-images
```

带 `systemd` 更新器的维护 FIT 需要再烧进闪存一次。从运行中的 Linux 通过 SSH 完成, 不用动跳线: `tools/flash-fit.sh` 上传文件, 通过 `/dev/mtdblock3` 写入, 回读并比对三个 MD5。如果 U-Boot 也重新编译过, 同一个脚本可以把 `u-boot.img` 写到 `/dev/mtdblock1`:

```
tools/flash-fit.sh <ip> tftp/fitImage          # mtd3, the FIT
tools/flash-fit.sh <ip> tftp/u-boot.img 1      # mtd1, U-Boot proper
```

✅ 检查点 脚本打印的三行 MD5 全部一致: 本地文件、上传结果和闪存回读。

7.4 冷启动

断电。拔掉 JTAG 适配器。上电。

⚠ 规则 留在 JTAG 口上的 FT232H 会把 TMS 保持在 3.3V, 并通过保护二极管让开发板的 3.3V 电源轨和闪存芯片在断电期间保持带电。一颗被 Linux 会话留在 4 字节地址模式的闪存芯片就此永远不会复位, 而使用 3 字节地址读取的 boot ROM 会停在 `0xffffffff28`。只有拔掉适配器, “重新上电”才算数。

🟢 **检查点** 大约一分钟内，无人登录的情况下，DHCP 服务器为 ax7020 保留的地址上 8000 端口有响应，可以送入一个比特流：

```
IP=$(curl -s http://10.42.0.1:8000/get_all_network_clients \
  | jq -r '.clients[] | select(.hostname=="ax7020" and .active_lease) | .ip')
curl http://$IP:8000/state
curl -F file=@bitstream/blinky/blinky.bit http://$IP:8000/bitstream
```

```
{"state": "operating", "sha256": "f69a32d2...", "bytes": 4045524,
  "idcode": "0x03727093", "filename": "blinky.bit"}
```

8 日常操作

8.1 找到开发板

U-Boot、维护内核和 API 内核各自申请自己的 DHCP 租约，所以一次启动过程中地址会变。按主机名查询 DHCP 服务器，而不是记住某个地址。上文使用的资产查询 API 是 [dhcp-inventory-api](#)；任何 DHCP 服务器的租约文件都可以用同样的方式处理。

8.2 SSH

根文件系统在 RAM 中，所以 dropbear 每次启动都会生成新的主机密钥。每次会话前：

```
ssh-keygen -R <ip>
```

启动后紧接着出现的 Connection refused 只说明 dropbear 还没起来；在这颗 CPU 上生成密钥大约需要 30 s。

8.3 回滚镜像

在镜像服务器上把 [ax7020-latest](#) 指向一个旧目录，然后给开发板重新上电。闪存中的内容不会有任何变化。

8.4 重写闪存

从运行中的 Linux 用 [tools/flash-fit.sh](#) 重写 FIT (mtd3) 和完整 U-Boot (mtd1)。[boot.bin](#) (mtd0) 只通过阶段 1 的 JTAG 流程写入，环境变量分区 (mtd2) 归 U-Boot 管。

8.5 备份闪存

从 U-Boot 通过 TFTP：

```
tools/ncsh.py <ip> "sf probe 0 30000000 0" "sf read 0x10000000 0 0x20000000" \
  "md5sum 0x10000000 0x20000000" "tftpput 0x10000000 0x20000000 ax7020-qspi.bin"
```

9 下一步

作为实验室系统，这条流水线已经完整。再加三项，它就能面向更大的网络，按此顺序：

1. API 的**身份认证**。带 AXI master 的比特流拥有对系统内存的完全访问权；加载未经审查的比特流，等同于多绕了几步的远程代码执行。
2. **清单中加入比特流**。第四行 `bitstream <file> <sha256>`，由服务在启动时加载，这样启动后逻辑阵列就不会是空的。
3. `/etc/ax7020-release` 中的**构建标记**，由 `/state` 报告，这样就能区分正在运行的镜像、闪存中的镜像和服务器上的镜像。

本文档中每一个决策背后的推理，以及那些最终变成琥珀色规则框的弯路，都在 maxclerkwell.tech/alinx 的五篇文章里（英文）：

- [Deploying Hardware Like Software: A Bitstream Pipeline for the Zynq](#)
- [ALINX AX7020 Bring-up: Mainline U-Boot Over JTAG, No FSBL, No Serial Cable](#)
- [Stages 2 & 3: A Yocto Linux in QSPI Flash That Fetches Its Own Updates](#)
- [Stages 4 & 5: An Open Bitstream Over REST, and the One Missing Line That Hangs Both Cores](#)
- [Stage 5 Complete: Cold Power-On to a Bitstream API, Nobody Logged In](#)

关于作者



施泓杰 (Stephan Bökelmann, MaxClerkwell)

工程师、物理学者、自由顾问 · 德国波鸿

施泓杰的工作贯穿工程技术的整个垂直栈：硅片级测量系统、嵌入式固件、PCB 设计与板级启动、FPGA 开发，以及其上的监控与数据基础设施。贯穿其中的主线是分布式数据采集：从物理系统中获得可靠的测量结果，无论这个系统是粒子探测器、生产线，还是一块不再响应的开发板。

他受过机械工与电工的职业培训，是国家认证机械工程师，拥有电气工程学士 (B.Eng.) 和计算机工程硕士 (M.Eng.) 学位，目前正在波鸿鲁尔大学 (Ruhr-Universität Bochum) 完成实验强子物理方向的博士研究，课题是 FAIR 的 PANDA 亮度探测器所用的 HV-MAPS 传感器。他是工程事务所 [nabla B](#) 的董事总经理、AUTO INTERN 的 COO 及其 skAlnet Edge-Compute 平台的系统架构师，并在波鸿工业大学 (THGA Bochum) 讲授编程与数据库课程。他是 emBO++ 的主要组织者、KiCon Europe 的联合组织者，曾在深圳 KiCon Asia 2025 上发表演讲，并持有多项测量技术专利。2007 年起从事专业工作。

承接项目

本教程所展示的这类工作正是我的本职，通过我在波鸿的公司 [nabla B](#) 承接：无厂商锁定的板级启动、基于 Yocto 与主线 U-Boot 的嵌入式 Linux、使用开源或厂商工具链的 FPGA 开发、从原理图到 EMC 的 KiCad PCB 设计，以及从传感器到服务器的数据采集系统。德语区 (DACH) 可现场服务，全球范围可远程协作，工作语言为英语或德语，从可行性研究直到量产产品。

- | | |
|------------|---|
| ✉ 项目委托 | office@nabla-b.engineering |
| ✉ 直接联系 | stephan@boekermann.net |
| ✉ 开发板厂商与合作 | collaboration@maxclerkwell.tech |
| 📁 服务与案例 | maxclerkwell.tech/hire |
| 🌐 LinkedIn | linkedin.com/in/accelerator-stephan |

博客与系列文章：[maxclerkwell.tech](#) • [maxclerkwell.tech/alinx](#)

本教程的源码：github.com/MaxClerkwell/ax7020-bringup

Discord, Full Stack Engineering: discord.gg/2BXuUY6hrX

© ⓘ 本文档以 [知识共享署名 4.0 \(CC BY 4.0\)](#) 许可协议发布：可自由使用、分享和改编，但须署名 Stephan Bökelmann (MaxClerkwell) 并附上 [maxclerkwell.tech/alinx](#) 的链接。AX7020 由 ALINX 赞助提供；厂商对其内容没有任何影响。